Advanced Data Structures

[CSE2010] Data Structures

Department of Data Science

Outline

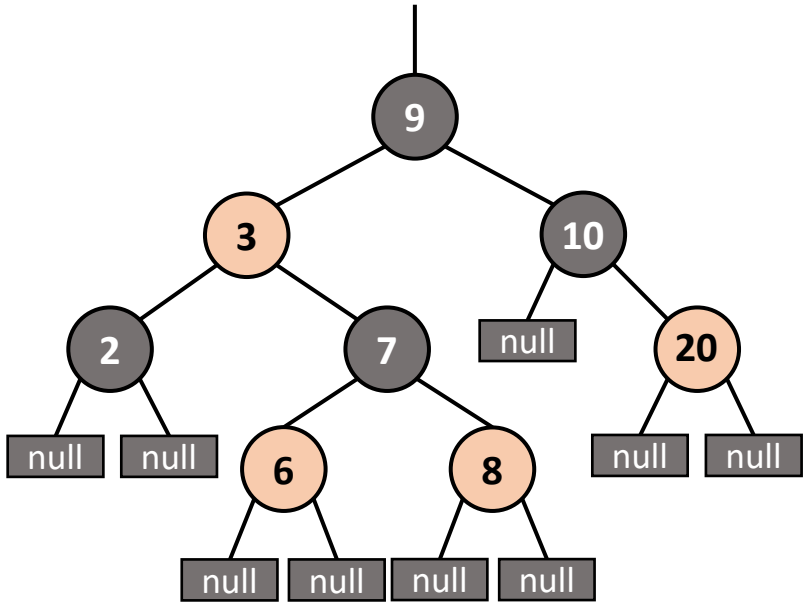
- Red-black trees
- Tries
- Bloom filters

Red-Black Trees

Red-Black Trees

- Self-balancing binary search tree
- Do not ensure strict balancing, unlike AVL trees
- Still ensure $O(logn)$ insertions, deletions, and retrievals.
- Require less memory and rebalance faster, so they are used in the situations where the tree will be modified frequently.

Properties of Red-Black Trees



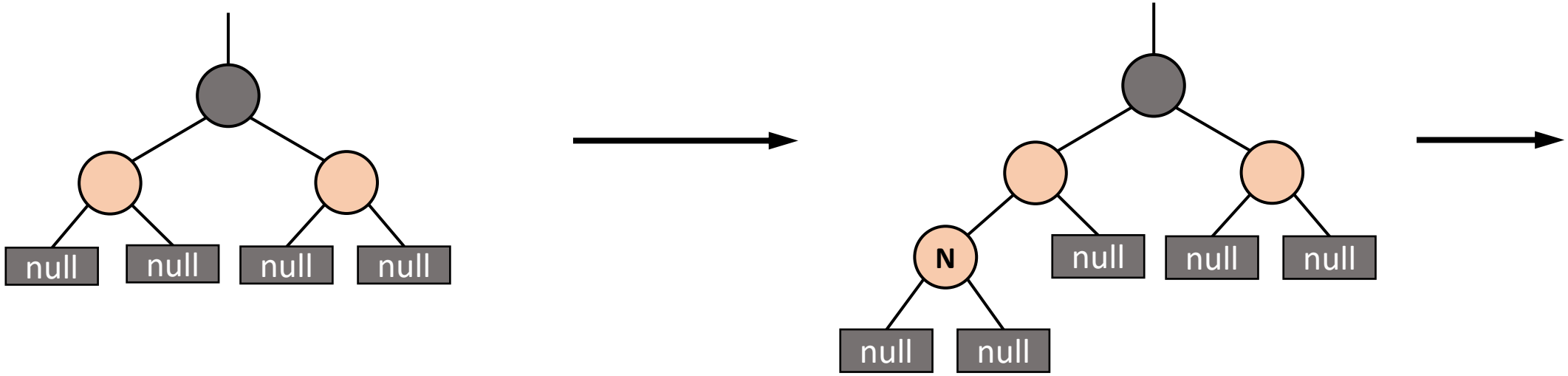
1. Every node is either red or black.
2. The root is black.
3. Leaves are black and NULL nodes.
4. Every red node must have two black children.
   - A red node cannot have red children (although a black node can have black children).
5. Every path from a node to a leaf has the same number of black nodes.

Why It Balances

- Two red nodes cannot be adjacent in a path (Property 4)
- No more than half the nodes in a path can be red.
- Given any two paths from a node to different leaves:
- The paths must have the same number b of black nodes (Property 5)
- Their red node counts are as different as possible:
  - Min-red path: the minimum number of red nodes is zero, so this path has b nodes total.
  - Max-red path: the maximum number of red nodes is b, since red nodes must have black children and there are b black nodes, so this path has $2b$ nodes total.
- The lengths of paths cannot differ by more than a factor of 2.
- Sufficient to ensure the $O(logn)$ runtime.

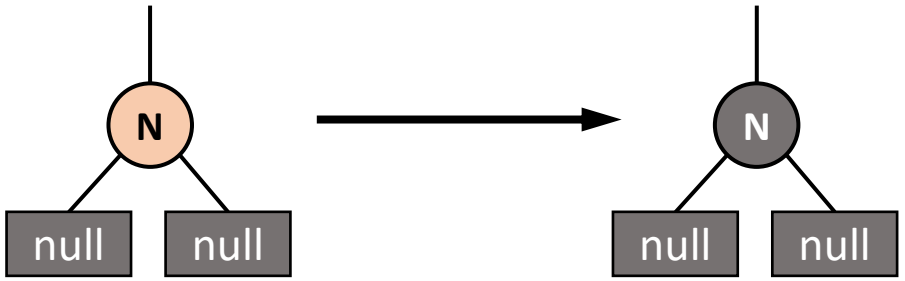
Insertion

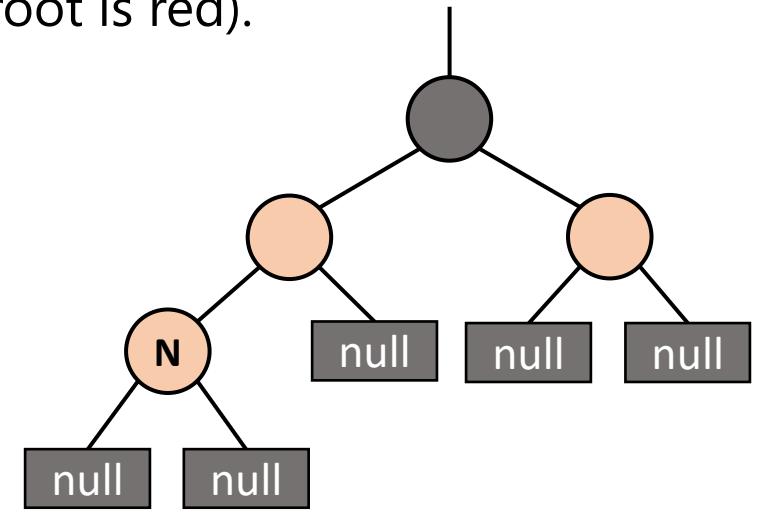
- First, begin with a typical binary search tree insertion
  - A new node is inserted at a leaf = it replaces a black NULL node
  - A new node is colored red and is given two black leaf (NULL) nodes.



Two Violations

1. Black violation: one path has more blacks than another path.
   - Impossible to happen, since the node just inserted is red.
2. Red violation: a red node has a red child (or the root is red).

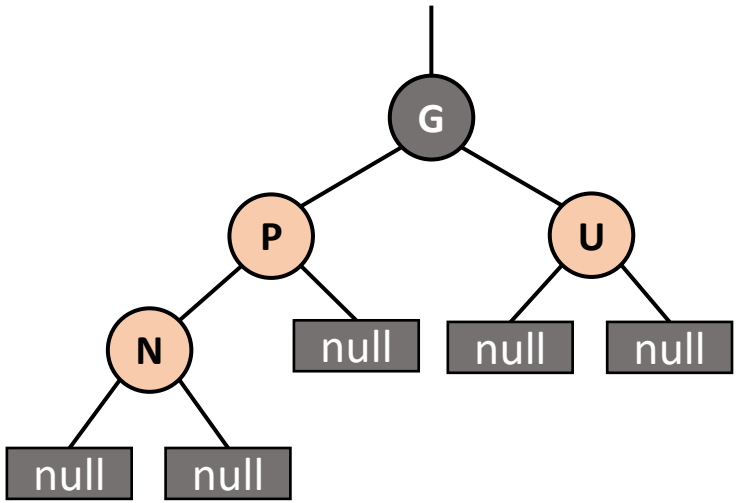




- If the root is red, turn it black to satisfy Property 2 (=the root is black)
- Otherwise: ?

Red Violations

- Known
  - Both N and P are red (red violation)
  - G is black (since we didn't previously have a red violation).
- Unknown
  - U could be either red or black.
  - U could be either a left or right child.
  - N could be either a left or right child.



- N: the current node
- P: N's parent
- G: N's grandparent
- U: N's uncle and P's sibling

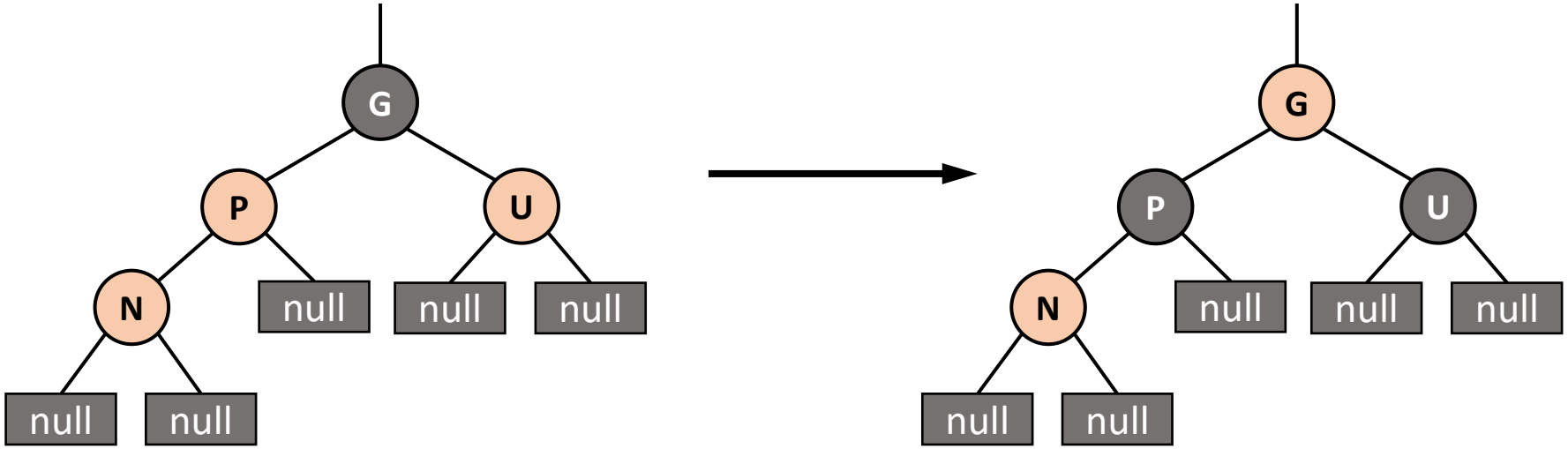
Different Cases of Red Violations

- Case 1. U is red

- Case 2. U is black
  A. P is G's left child. N is P's left child (LL)
  B. P is G's left child. N is P's right child (LR)
  C. P is G's right child. N is P's right child (RR)
  D. P is G's right child. N is P's left child (RL)

Case 1: U is Red

- Toggle the colors of P, U and G:
  Flip G from black to red, and flip P and U from red to black.
- Making G red may create a red violation with G's parent. Then we recursively apply the full logic to handle a red violation, where G becomes new N.



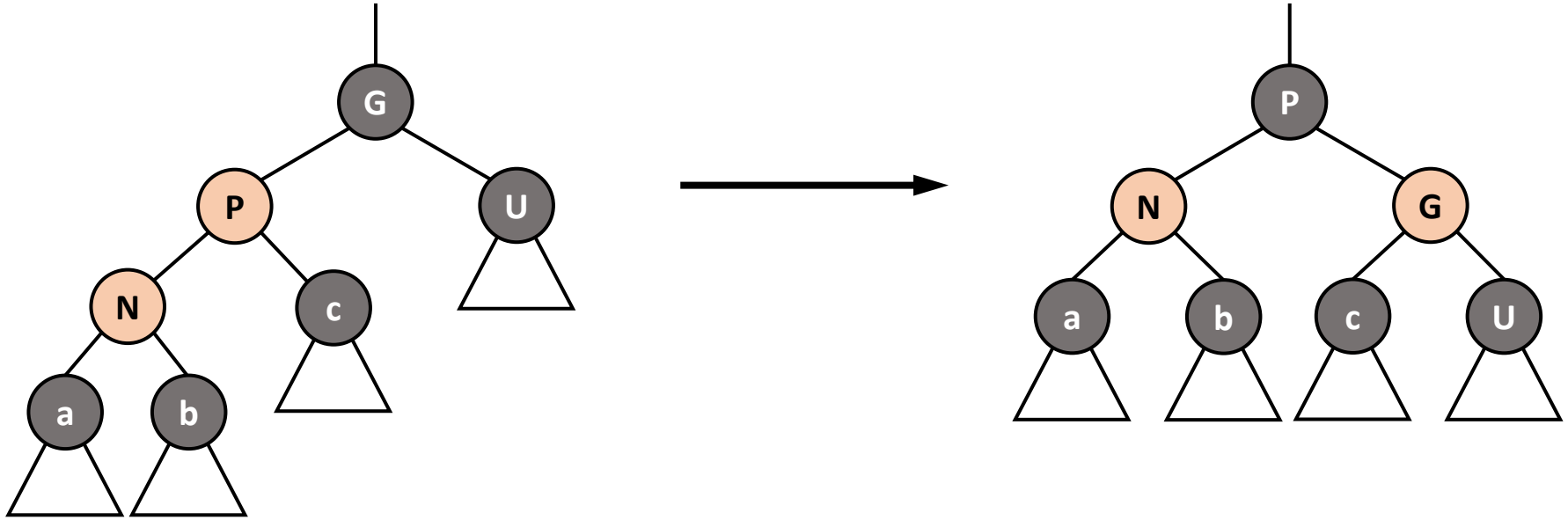
Case 2: U is Black

- Fix up the red violation without:
  1. messing up the ordering of the binary search tree
  2. introducing a black violation (more black nodes on one path than another)

- In the four scenarios (LL, LR, RR, RL):
  - the red violation is fixed with rotations that maintain the node ordering.
  - rotations maintain the exact number of black nodes in each path.

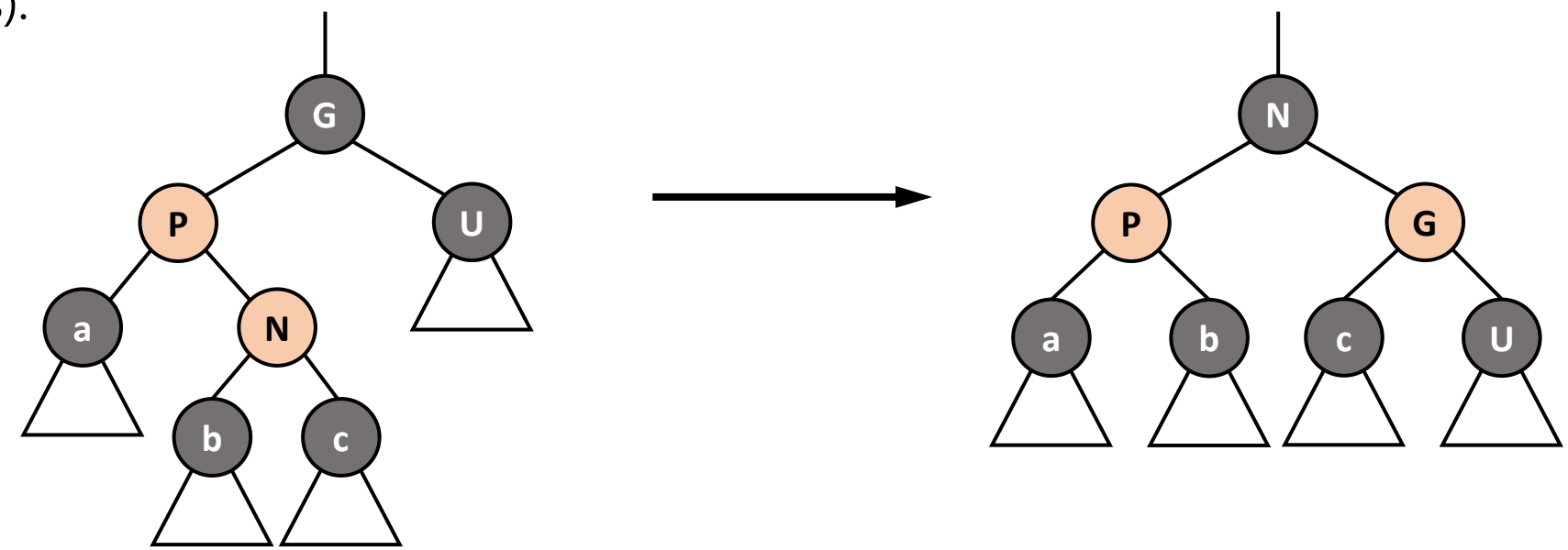
Case 2A: U is Black and LL

- Rotation maintains the node ordering ($a \leq N \leq b \leq P \leq c \leq G \leq U$)
- Tree maintains the same number of black nodes in the path down to each subtree a, b, c, and U.



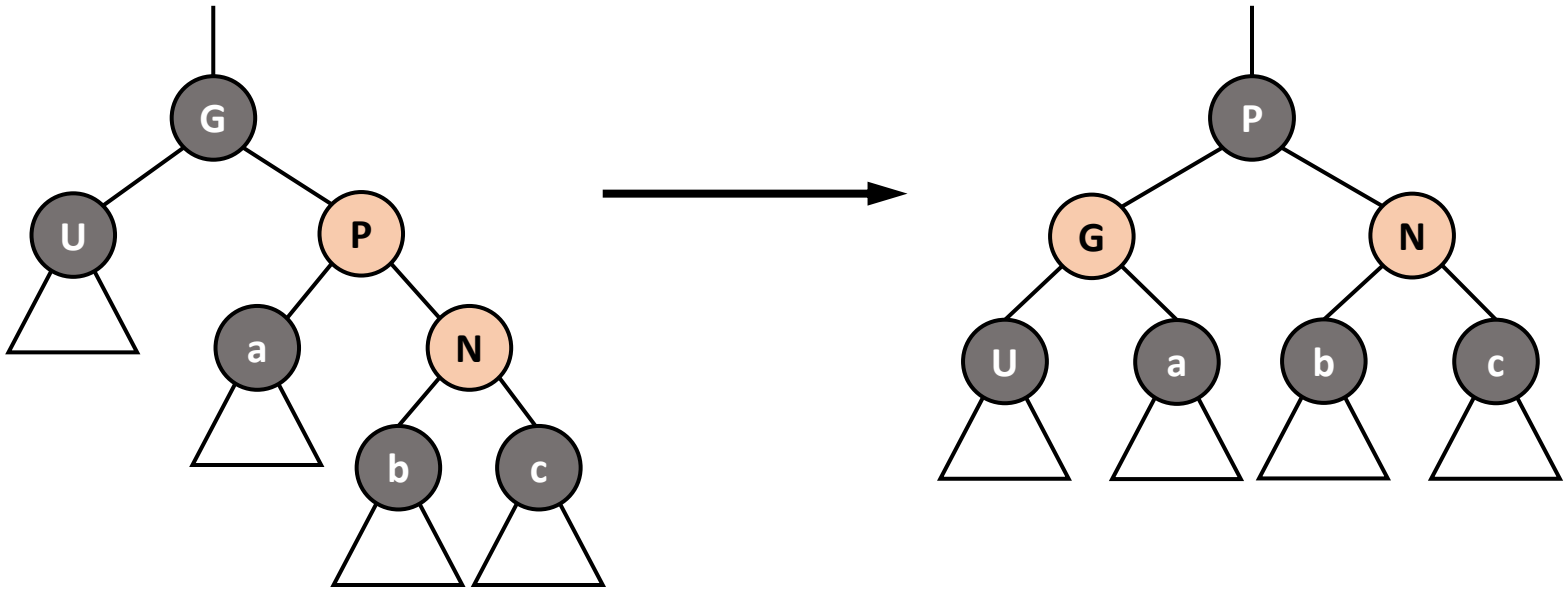
Case 2B: U is Black and LR

- The rotations in Case B resolve the red violation and maintain the in-order property: $a \leq P \leq b \leq N \leq c \leq G \leq U$.
- The count of the black nodes remains constant in each path down to the leaves (or subtrees).



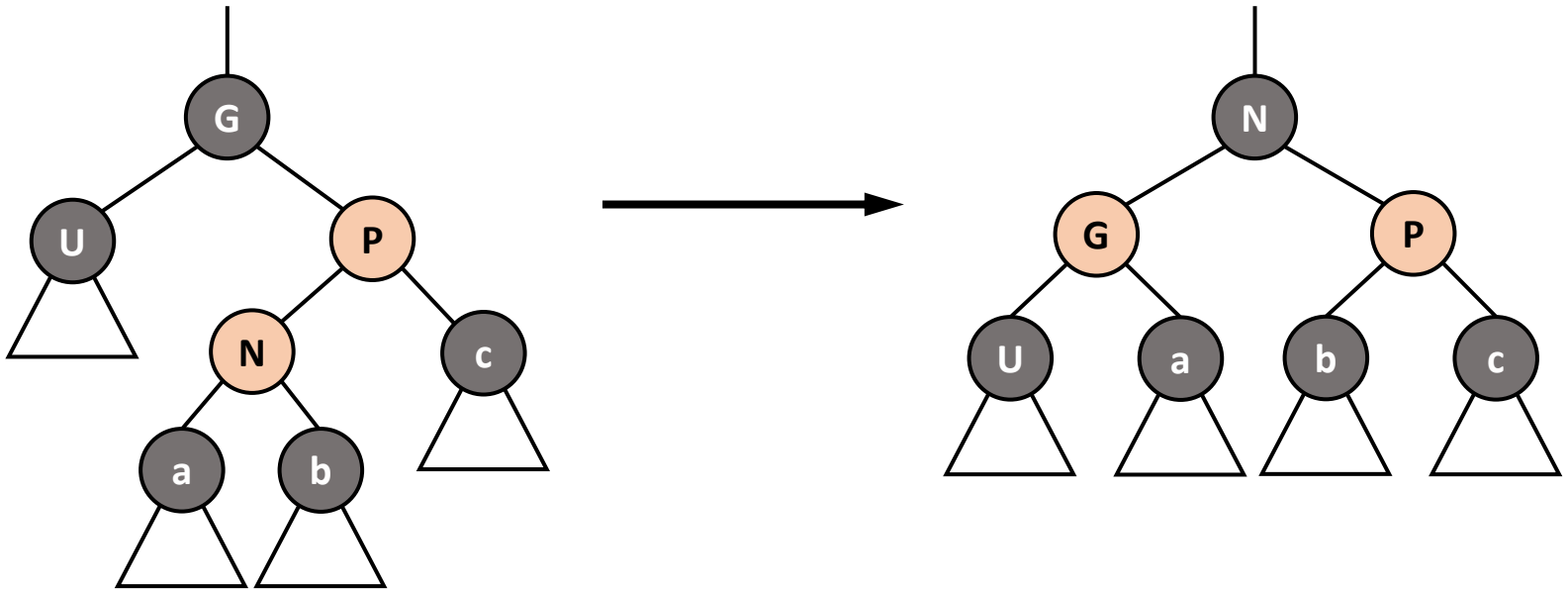
Case 2C: U is Black and RR

- Mirror image of case 2A



Case 2D: U is Black and RL

- Mirror image of case 2B



Time Complexity

Operation	Running Time
Search	O(logn)
Insertion	O(logn)
Deletion	O(logn)

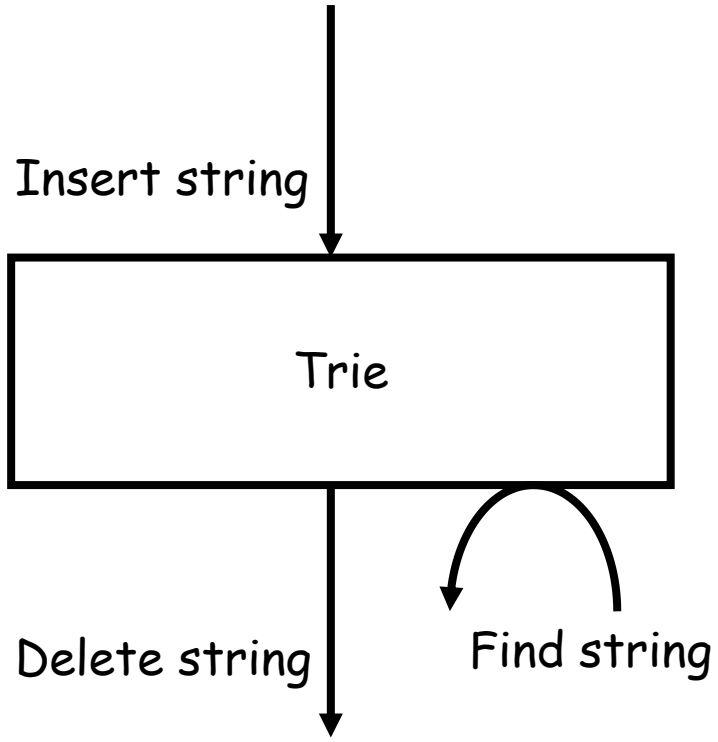
- Time complexity is O(1) per level, broken into a down phase and an up phase.
  - Down phase: searching
  - Up phase: maintaining the exact number of black nodes in a path

Tries (Prefix Trees)

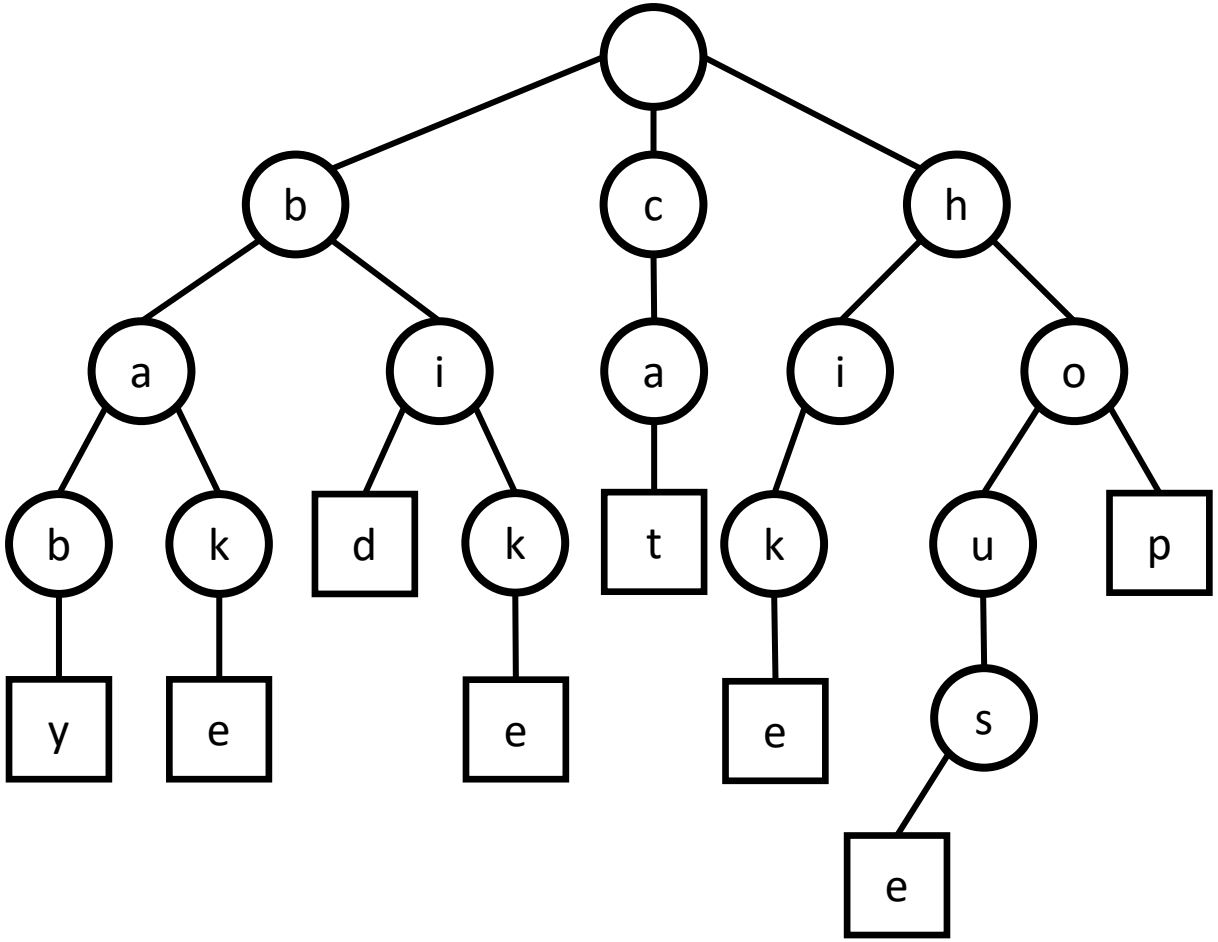
Tree-based data structure for storing strings in order to support fast pattern matching

Tries (Prefix Trees)

- Tree-based data structure for storing strings in order to support fast pattern matching
  - re**trie**val
- Inserts, finds, deletes a word in $O(l)$ time where l represents the length of the word.
  - Faster than hash tables & binary search trees depending on the application



Trie



- Standard trie for a set S of strings from alphabet Σ
- Each node except the root is labeled with a character of Σ
- The children of an internal node have distinct labels.
- A leaf represents a string of S
  - Concatenation of the labels of the nodes on the path from the root to a leaf yields the string of S

Applications

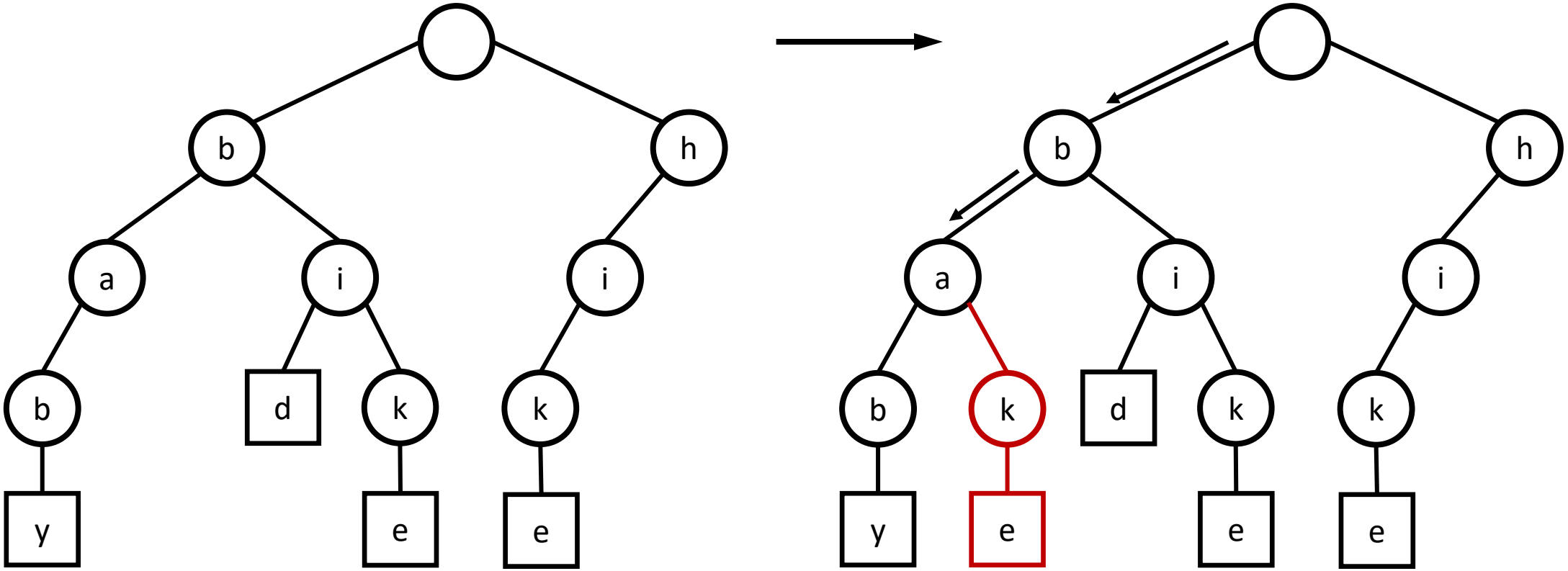
- Count the total number of words (starting with a given prefix)
- Sort all words
- Enumerate the words (starting with the same prefix)

Properties of Tries

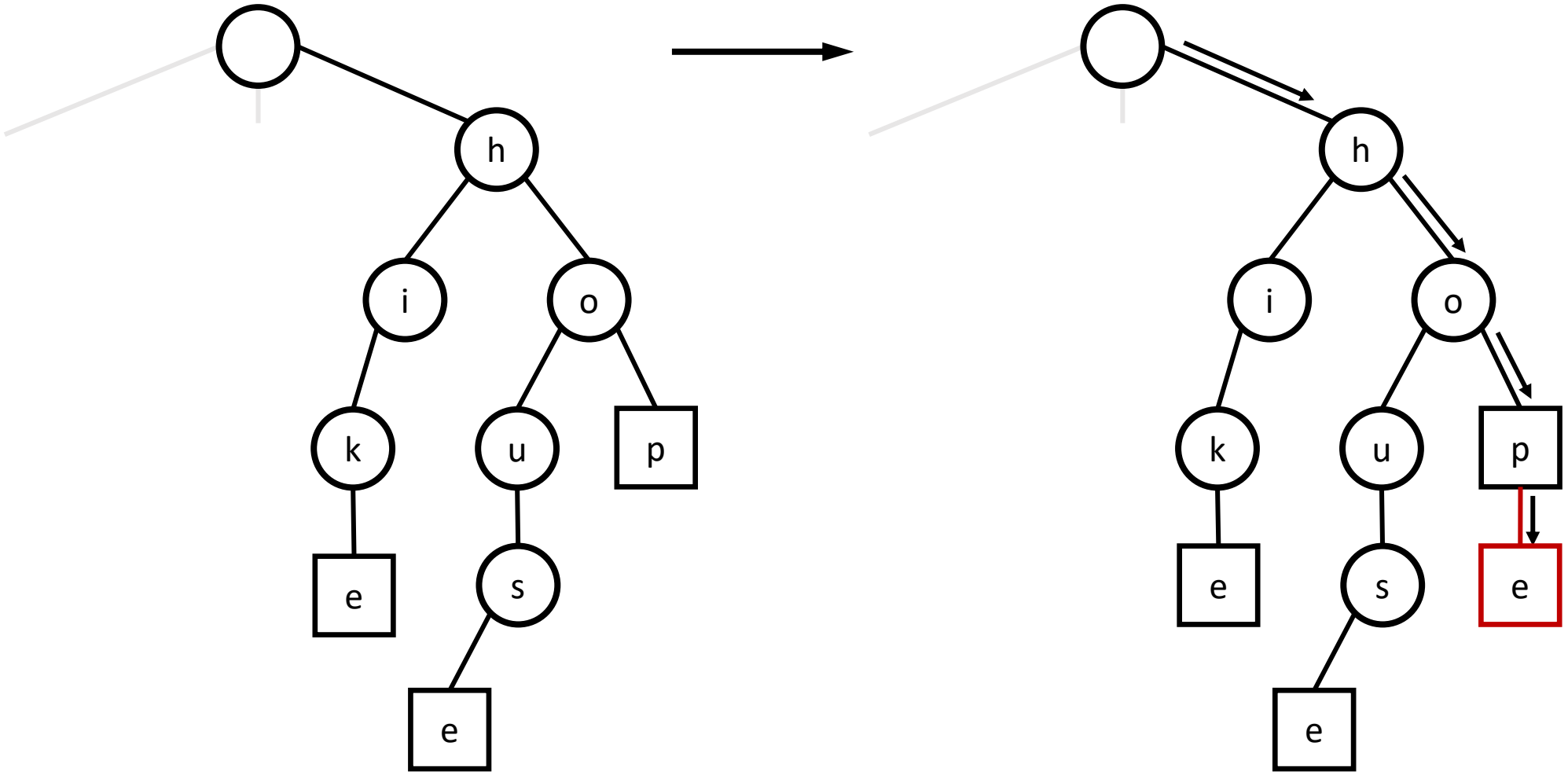
- Given a trie T storing a collection S of strings of total length n from an alphabet Σ:
- The height of T is equal to the length of the longest string in S.
- Every internal node of T has at most $|\Sigma|$ children.
- Trie T has at most $|S|$ leaves.
- The number of nodes of T is at most $n + 1$.

Insertion Example: “bake”

1. From the root, visit a node that corresponds to each char if such node exists
2. Construct new nodes of the non-existing chars, and mark end of the word for the last node.

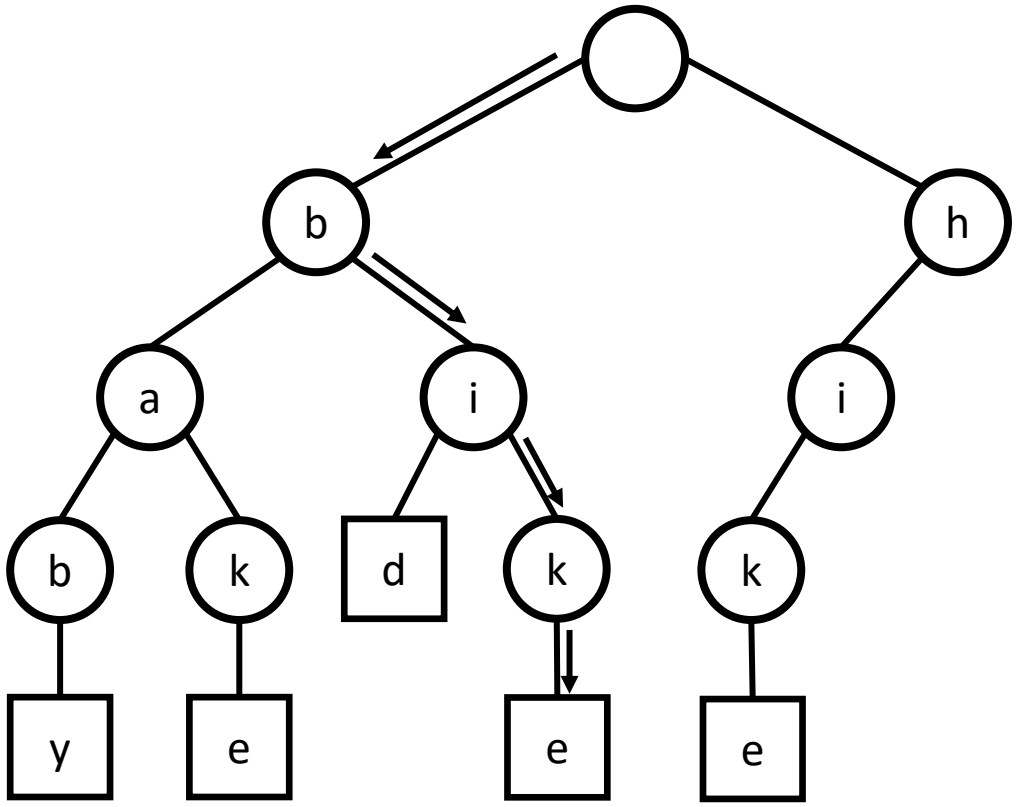


Insertion Example: “hope”



Search Example: “bike”

1. From the root, visit the node associated with each char in a word.
2. If there exists a path from the root the node associated with the word, return true.
3. Otherwise, return false.



Trie Implementation (Python)

```
ENDS_HERE = '#'
class Trie:
  def __init__(self):
    self.dic = {}

  def insert(self, text):
    node = self.dic
    for ch in text:
      if ch not in node:
        node[ch] = {}
      node = node[ch]
    node[ENDS_HERE] = True
```

```
  def find(self, text):
    node = self.dic
    for ch in text:
      if ch not in node:
        return False
      else:
        node = node[ch]
    return ENDS_HERE in node
```

- Trie node has
  - a dictionary of (char, child's dictionary) pairs
- Running time: $O(l)$
  - where l is the length of the string

TrieNode Implementation (C++)

- Trie node has:
  - data
  - endsHere
  - a collection of child nodes

```
const int CHAR_SIZE = 128;
class TrieNode {
public:
  char data;
  bool endsHere;
  TrieNode* child[CHAR_SIZE];
  TrieNode(): data(0), endsHere(false), child() {}
  TrieNode(char c): data(c), endsHere(false), child() {}
  void insert(const char*);
  bool search(const char*);
};
```

Insertion & Search Implementation

```
void TrieNode::insert(const char* word) {
  TrieNode* root = this;
  for (int i = 0; word[i]; ++i) {
    if (!root->child[word[i]])
      root->child[word[i]] = new TrieNode(word[i]);
    root = root->child[word[i]];
  }
  root->endsHere = true;
}
```

```
bool TrieNode::search(const char* word) {
  TrieNode* root = this;
  for (int i = 0; word[i]; ++i) {
    if (!root->child[word[i]])
      return false;
    root = root->child[word[i]];
  }
  return root->endsHere;
}
```

- Running time: $O(l)$
  - where l is the length of the string

Takeaways

- Advantage
  - Fast
- Disadvantage
  - Much memory to store strings
    (each node needs too many node pointers while the occupancy of each node is less)
- Enhancement
  - A compressed trie is similar to a standard trie but it ensures that each internal node in the trie has at least two children.

Time Complexity

- $O(l)$ time to insert, search for a string with length l.

Operation	Average case	Worst Case
Insertion	$O(l)$	$O(l)$
Search	$O(l)$	$O(l)$

Exercise: Autocomplete System

- **Problem**
  - Implement an autocomplete system. Given a query string s and a set of all possible query strings, return all strings in the set that have s as a prefix.
- Example
  - Input: words=**[cat, dog, deer, deal]**, s=**de**
  - Output: **[deer, deal]**

Naïve Solution

```
def naiveAutocomplete(words, s):
  out = []
  for word in words:
    if word.startswith(s):
      out.append(word)
  return out
```

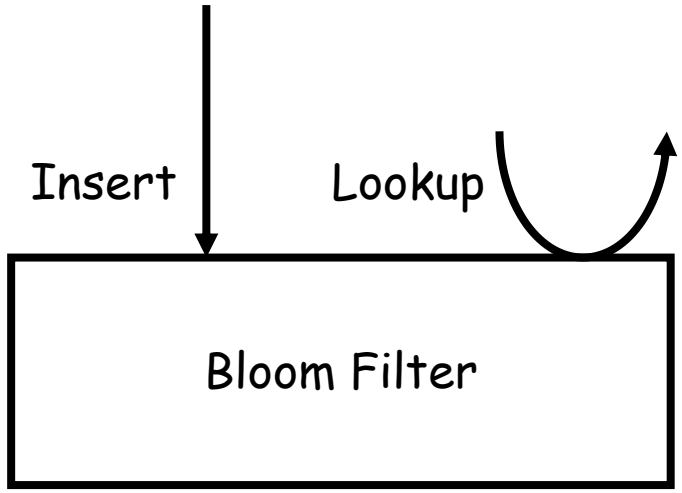
- $O(n)$ time where n is the number of words
- Can we do better than this on average?

Bloom Filters

Space-efficient probabilistic data structure to check whether an element is present in a set

Bloom Filters

- Tell that the element either definitely is **not** in the set or **may** be in the set
- Main operations
  - insert(x): insert an element x
  - lookup(x): check whether an element is present in the set with a false positive probability
- Implementation
  - a bit array as data structure
  - multiple hash functions

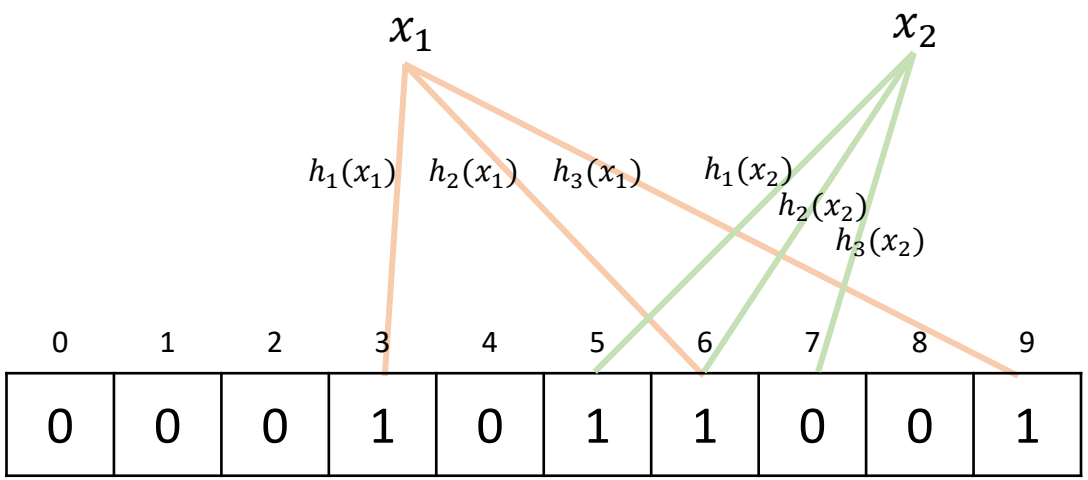


Bloom Filters

- Given a **bit array** with m bits and k **hash functions** $h_1(x)$, $h_2(x)$, …, $h_k(x)$
- Tell if x is present in a set

1. x may be present if bits at k slots $h_1(x)$, $h_2(x)$, …, $h_k(x)$ are 1
2. x does not exist otherwise

- Bit array with 10 bits and 3 hash functions



x_1 **may** exist in a set

x_2 does not exist in a set

Insertion

- Start off with a bit array initialized to zero.

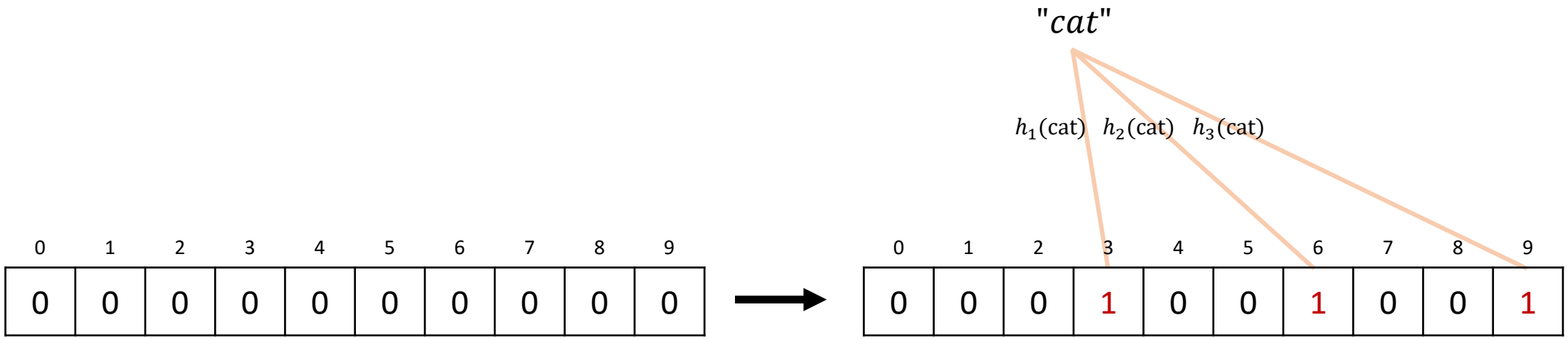
0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

- For every element to be inserted

1. Apply k different hash functions and treat the resulting k values as indices in the array
2. Set each of the k array elements to 1.

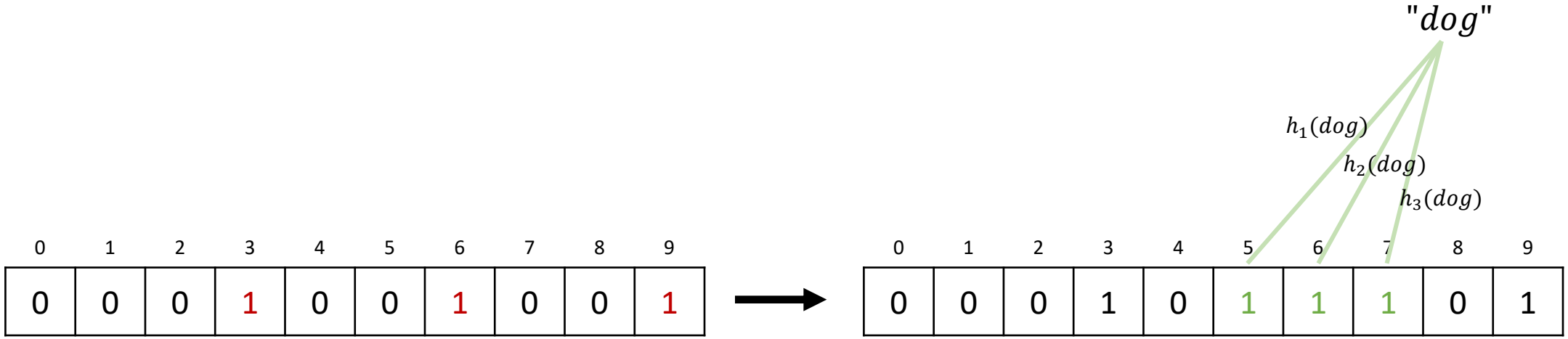
Insertion Example

- Suppose we insert "cat" to the empty filter.
  - $h_1(\text{“}cat\text{”}) = 3$
  - $h_2(\text{“}cat\text{”}) = 6$
  - $h_3(\text{“}cat\text{”}) = 9$



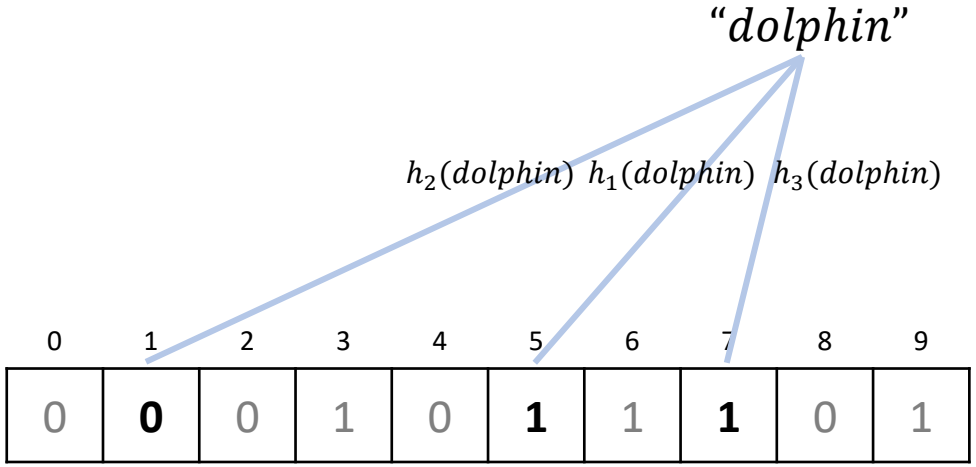
Insertion Example

- Suppose we insert "dog" to the filter.
  - $h_1(\text{“}dog\text{”}) = 5$
  - $h_2(\text{”}dog\text{”}) = 6$
  - $h_3(\text{”}dog\text{”}) = 7$



Negative Lookup Example

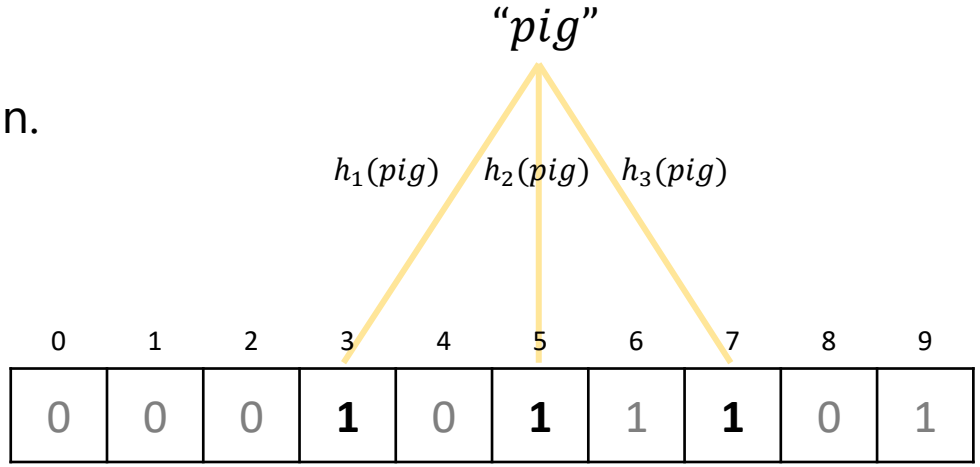
- Lookup "$dolphin$" in the filter.
  - $h_1(\text{“}dolphin\text{”}) = 5$
  - $h_2(\text{“}dolphin\text{”}) = 1$
  - $h_3(\text{“}dolphin\text{”}) = 7$



$\boldsymbol{dolphin}$ **is not in a set!**

False Positive Lookup Example

- Lookup "pig" in the filter.
  - $h_1(\text{“}pig\text{”}) = 3$
  - $h_2(\text{“}pig\text{”}) = 5$
  - $h_3(\text{“}pig\text{”}) = 7$
- Bloom filter erroneously claim that "pig" is present.
- Might be huge downside depending on the application.



False positive!

Properties of Bloom Filters

- Adding an element never fails.
- False positive may occur.
  - The false positive rate increases as elements are added until all bits in the filter are set to 1. Then all queries yield a positive result.
- False negative never occurs.

Probability of False Positives

- Probability of all the k bits computed by the hash functions is 1

$$P = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^{k}$$

  - m: the size of bit array
  - k: the number of hash functions
  - n: the number of expected elements to be inserted to the filter
- Probability that a certain bit is not set to 1 by a certain hash function during the insertion: $1 - \frac{1}{m}$
- Probability that the bit is not set to 1 by any of k hash functions: $\left(1 - \frac{1}{m}\right)^{k}$
- Probability that the bit is still 0 if we have inserted n elements: $\left(1 - \frac{1}{m}\right)^{kn}$
- Probability that the bit is 1 after the insertions: $1 - \left(1 - \frac{1}{m}\right)^{kn}$

Takeaways

- Benefits
  - Space–efficient: a bloom filter of a fixed size can represent a set with a large number of elements.
  - Fast: $O(k)$ time to insert and lookup (where k is the number of hash functions)
- Downsides
  - False positive, but controllable
  - Deleting elements is not possible.
    - Deleting a single element by clearing bits at slots generated by k hash functions may cause the deletion of other elements.